UQFFLearningAssessment_002.cpp

UQFF C++ Source Module

`UQFFLearningAssessment_002.cpp ■■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ UQFF Learning Assessment — PhD Research Edition (v1.0) Systems [0]–[7]: [0] W2_TRIADIC — Westerlund 2, DPM 4-Component Buoyancy Integral [1] SGRA_BUOY — Sgr A* SMBH, Buoyancy-Dominant DPM [2] PILLARS_DPM — Pillars of Creation, DPM Resonance ×10■ [3] RESONANCE_26D — Abstract 26-Layer Vacuum Resonance Framework`

Daniel T. Murphy • UQFF Research Consortium • 2026

Lines: 1,086 Size: 60,987 bytes

Module Description

UQFFLearningAssessment_002.cpp

■■■ ■■■■■■■■■■■■■■■■■■

UQFF Learning Assessment — PhD Research Edition (v1.0)

Systems [0]–[7]:

[0] W2_TRIADIC — Westerlund 2, DPM 4-Component Buoyancy Integral

[1] SGRA_BUOY — Sgr A* SMBH, Buoyancy-Dominant DPM

[2] PILLARS_DPM — Pillars of Creation, DPM Resonance ×10■

[3] RESONANCE_26D — Abstract 26-Layer Vacuum Resonance Framework

[4] UTE2_SC — UTe2 Topological Superconductor, δ_n Series

[5] ANYONS_CERN — Anyon Condensate (CERN 2025), Non-unitary TQFT

[6] H_RES_FE56 — Fe-56 Nuclear Shell, H_res Periodic Table Model

[7] VAC_HARMONICS — Vacuum Density Series + Buoyancy Harmonics U_g2

Audience : PhD candidates in quantum physics, astrophysics, cosmology.

Prereq : UQFFLearningAssessment_001.cpp (Research Student Edition).

Source : grok_share_c020496d9e.txt + Daniel T. Murphy UQFF manuscript v4.80.

Author : Encoded by Grok (xAI), based on Daniel T. Murphy's UQFF manuscript.

Date : 2026

Depends : UQFF_LA002_Constants.h (must be in same include path)

Key features (PhD Edition beyond _001.cpp):

1. DPM 4-component buoyancy integral: F_U_Bi_i ≈ ±10^{208..211} N

2. Full 26-layer resonance sum R(t) = Σ■■■^{2}■ [...]

3. UTe2 topological SC δ_n series (n = 1..9, f_topo = 0.20)

4. Anyon F_UBii with Gaussian collapse $\exp(-\delta_c^2/2\sigma^2)$

5. H_res nuclear shell model Z=1..118, Fe-56 reference

6. Vacuum Density Series Σ(1/n^2■)·[SSq]^n ≈ Li■■([SSq]) ≈ 0.570

7. Buoyancy Harmonics U_g2 = Σ H_m(1−e^{−[SSq]m})cos(ω t_n)

8. Full U_m form with P_SCm, E_react, f_Heaviside, f_quasi, e^{−[SSq]}

9. Dynamic [SSq](n,t) = log(ρ_SCm/ρ_UA)·n·e^{−(π−t_n)}

10. Scale range: 1 fm (nuclear) → 1 Mpc (cosmic void) = 37 log-decades

Integration:

- Header-only (include guard pattern matching _001.cpp)

- #include "UQFFLearningAssessment_002.cpp" from MAIN_1_CoAnQi.cpp

- All UQFF 2.0 Self-Expanding Framework features present:

setEnableLogging(), setDynamicParameter(), getDynamicParameter(),

exportState(), cross_validate<OtherModuleT>()

Reference files:

UQFF_LA002_Constants.h — PhD constants (F_rel, SSq, E_LEP, ...)

UQFF_LA002_SystemParams.md — Curriculum outline + learning objectives

UQFF_LA002_PhysicsNotes.md — Full equation reference sheet

Source Code

```
/**
 * UQFFLearningAssessment_002.cpp
 * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * UQFF Learning Assessment — PhD Research Edition (v1.0)
 *
 * Systems [0]–[7]:
 *   [0] W2_TRIADIC       — Westerlund 2, DPM 4-Component Buoyancy Integral
 *   [1] SGRA_BUOY        — Sgr A* SMBH, Buoyancy-Dominant DPM
 *   [2] PILLARS_DPM      — Pillars of Creation, DPM Resonance ×10■
 *   [3] RESONANCE_26D    — Abstract 26-Layer Vacuum Resonance Framework
 *   [4] UTE2_SC          — UTe2 Topological Superconductor, δ_n Series
 *   [5] ANYONS_CERN      — Anyon Condensate (CERN 2025), Non-unitary TQFT
 *   [6] H_RES_FE56       — Fe-56 Nuclear Shell, H_res Periodic Table Model
 *   [7] VAC_HARMONICS    — Vacuum Density Series + Buoyancy Harmonics U_g2
 *
 * Audience  : PhD candidates in quantum physics, astrophysics, cosmology.
 * Prereq    : UQFFLearningAssessment_001.cpp (Research Student Edition).
 * Source    : grok_share_c020496d9e.txt + Daniel T. Murphy UQFF manuscript v4.80.
 * Author    : Encoded by Grok (xAI), based on Daniel T. Murphy&#39;s UQFF manuscript.
 * Date      : 2026
 * Depends   : UQFF_LA002_Constants.h  (must be in same include path)
 *
 * Key features (PhD Edition beyond _001.cpp):
 *   1. DPM 4-component buoyancy integral: F_U_Bi_i ≈ ±10^{208..211} N
```

```
 *   2. Full 26-layer resonance sum R(t) = Σ■■■²■ [...]
 *   3. UTe2 topological SC δ_n series  (n = 1..9, f_topo = 0.20)
 *   4. Anyon F_UBii with Gaussian collapse exp(−δ_c²/2σ²)
 *   5. H_res nuclear shell model Z=1..118, Fe-56 reference
 *   6. Vacuum Density Series  Σ(1/n²■)·[SSq]^n  ≈  Li■■([SSq]) ≈ 0.570
 *   7. Buoyancy Harmonics  U_g2 = Σ H_m(1−e^{−[SSq]m})cos(ω t_n)
 *   8. Full U_m form with P_SCm, E_react, f_Heaviside, f_quasi, e^{−[SSq]}
 *   9. Dynamic [SSq](n,t) = log(ρ_SCm/ρ_UA)·n·e^{−(π−t_n)}
 *  10. Scale range: 1 fm (nuclear) → 1 Mpc (cosmic void) = 37 log-decades
 *
 * Integration:
 *   - Header-only (include guard pattern matching _001.cpp)
 *   - #include &quot;UQFFLearningAssessment_002.cpp&quot; from MAIN_1_CoAnQi.cpp
 *   - All UQFF 2.0 Self-Expanding Framework features present:
 *       setEnableLogging(), setDynamicParameter(), getDynamicParameter(),
 *       exportState(), cross_validate&lt;OtherModuleT&gt;()
 *
 * Reference files:
 *   UQFF_LA002_Constants.h      — PhD constants (F_rel, SSq, E_LEP, ...)
 *   UQFF_LA002_SystemParams.md  — Curriculum outline + learning objectives
 *   UQFF_LA002_PhysicsNotes.md  — Full equation reference sheet
 */

#ifndef UQFF_LEARNING_ASSESSMENT_002_H
#define UQFF_LEARNING_ASSESSMENT_002_H

#include &lt;iostream&gt;
#include &lt;cmath&gt;
#include &lt;iomanip&gt;
#include &lt;string&gt;
#include &lt;limits&gt;
#include &lt;map&gt;
#include &lt;fstream&gt;
#include &lt;array&gt;
#include &lt;algorithm&gt;
#include &lt;numeric&gt;

#include &quot;UQFF_LA002_Constants.h&quot;   // PhD-edition constants (namespace UQFF_LA002)

static constexpr double UQFF_LA002_PI     = 3.14159265358979323846;
static constexpr double UQFF_LA002_TWO_PI = 6.28318530717958647692;

// ■■ System index constants ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
static constexpr int SYS_W2_TRIADIC    = 0;  ///&lt; Westerlund 2 — DPM 4-component
static constexpr int SYS_SGRA_BUOY     = 1;  ///&lt; Sgr A* SMBH — buoyancy-dominant
static constexpr int SYS_PILLARS_DPM   = 2;  ///&lt; Pillars of Creation — DPM resonance
static constexpr int SYS_RESONANCE_26D = 3;  ///&lt; Abstract 26-layer resonance
static constexpr int SYS_UTE2_SC       = 4;  ///&lt; UTe2 topological superconductor
static constexpr int SYS_ANYONS_CERN   = 5;  ///&lt; Anyon condensate (CERN 2025)
static constexpr int SYS_H_RES_FE56    = 6;  ///&lt; Fe-56 nuclear shell / H_res
static constexpr int SYS_VAC_HARMONICS = 7;  ///&lt; Vacuum density series + harmonics
static constexpr int NUM_SYSTEMS_002   = 8;

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class UQFFLearningAssessment002 {
```

```cpp
private:

    // ■■ Shared physical constants ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    const double G            = 6.6743e-11;   // N m² kg■²
    const double c_light      = 2.998e8;       // m s■¹
    const double hbar         = 1.0546e-34;    // J s
    const double Lambda       = 1.1e-52;       // m■²
    const double M_sun        = 1.989e30;      // kg
    const double m_e_kg       = 9.109e-31;     // kg  (electron)
    const double m_p_kg       = 1.6726e-27;    // kg  (proton)
    const double m_u_kg       = 1.66054e-27;   // kg  (atomic mass unit)
    const double mu_B_SI      = 9.274e-24;     // J T■¹  (Bohr magneton)
    const double h_planck     = 6.626e-34;     // J s
    const double t_Hubble     = 4.355e17;      // s  (13.8 Gyr)

    // ■■ UQFF standard buoyancy pipeline ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    const double beta_i       = 0.61;          // dimensionless
    const double omega_g      = 7.3e-16;       // rad s■¹
    const double U_UA         = 1.0e-11;       // dimensionless
    const double f_TRZ        = 0.1;           // TRZ fraction
    const double rho_vac_UA   = 7.09e-36;      // J m■³
    const double rho_vac_SCm  = 7.09e-37;      // J m■³
    const double B_crit       = 4.4e13;        // T

    // ■■ PhD edition constants ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    const double F_rel        = 4.31e33;       // N   (2024 LEP calibration)
    const double E_LEP        = 3.204e-8;      // J   (200 GeV)
    const double Q_wave       = 1.0e12;        // coupling amplifier
    const double SSq          = 0.57;          // [SSq] index
    const double gamma_d      = 5.787e-10;     // s■¹ (= 5e-5/86400)
    const double delta_k_eta  = 7.25e8;        // nuclear binding differential
    const double k_Ub_val     = 0.1;           // buoyancy calibration
    const double f_Ub_cal     = 2.20e7;        // calibrated f_Ub
    const double delta_c_any  = 1.686;         // anyon collapse threshold
    const double sigma_any    = 1.0;           // TQFT variance
    const double k_A_hres     = 1.0e-3;        // H_res amplitude k_A
    const double A_H_ref      = 1.0;           // hydrogen mass number reference
    const double k0_nuc       = 0.1;           // nuclear coupling baseline

    // ■■ Assessment metrics ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    double diversity_score    = 1.0;   // 8/8 physical regimes
    double dynamic_score      = 1.0;   // 10 unique PhD processes
    double scalability_score  = 0.0;   // log-decades / 40 (computed at init)
    double coverage_score     = 1.0;   // 8/8 systems computed

    // ■■ UQFF 2.0 Self-Expanding Framework ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    bool                        logging_enabled = false;
    std::map&lt;std::string,double&gt; dynamic_params;

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // Per-system parameters
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

    // [0] Westerlund 2 — DPM 4-component triadic
    double M_w2t, r_w2t, DPM_res_w2, F_LENR_w2, T_sf_w2;
```

```
    double M_ext_w2t, r_ext_w2t;

    // [1] Sgr A* — buoyancy-dominant DPM
    double M_sgra_b, r_sgra_b, DPM_res_sgra, F_LENR_sgra, T_sf_sgra;
    double M_ext_sgra_b, r_ext_sgra_b;

    // [2] Pillars of Creation — DPM resonance ×10■
    double M_pil_d, r_pil_d, DPM_res_pil, F_LENR_pil, T_sf_pil;
    double M_ext_pil_d, r_ext_pil_d;

    // [3] 26D resonance layer framework
    double M_res26, r_res26, N_layers_res26, T_sf_res26, SSq_res26;
    double M_ext_res26, r_ext_res26;

    // [4] UTe2 topological superconductor
    double M_ute2, r_ute2, B_ute2, f_topo_ute2;
    double M_ext_ute2, r_ext_ute2;

    // [5] Anyon condensate (CERN 2025)
    double M_any, r_any, E_anyons_loc, delta_c_loc, sigma_loc;
    double M_ext_any, r_ext_any;

    // [6] H_res nuclear shell model — Fe-56
    double M_fe56, r_fe56, Z_fe56, A_fe56, N_fe56;
    double M_ext_fe56, r_ext_fe56;

    // [7] Vacuum density series + buoyancy harmonics
    double M_vac, r_vac, N_harmonics_vac, gamma_vac_loc;
    double M_ext_vac, r_ext_vac;

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // Private helpers
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

    double g_base_for(int idx) const {
        switch (idx) {
            case SYS_W2_TRIADIC:    return G * M_w2t    / (r_w2t    * r_w2t);
            case SYS_SGRA_BUOY:     return G * M_sgra_b / (r_sgra_b * r_sgra_b);
            case SYS_PILLARS_DPM:   return G * M_pil_d  / (r_pil_d  * r_pil_d);
            case SYS_RESONANCE_26D: return G * M_res26  / (r_res26  * r_res26);
            case SYS_UTE2_SC:       return G * M_ute2   / (r_ute2   * r_ute2);
            case SYS_ANYONS_CERN:   return G * M_any    / (r_any    * r_any);
            case SYS_H_RES_FE56:    return G * M_fe56   / (r_fe56   * r_fe56);
            case SYS_VAC_HARMONICS: return G * M_vac    / (r_vac    * r_vac);
            default:                return 0.0;
        }
    }

    double get_M(int idx) const {
        switch (idx) {
            case SYS_W2_TRIADIC:    return M_w2t;
            case SYS_SGRA_BUOY:     return M_sgra_b;
            case SYS_PILLARS_DPM:   return M_pil_d;
            case SYS_RESONANCE_26D: return M_res26;
            case SYS_UTE2_SC:       return M_ute2;
```

```
            case SYS_ANYONS_CERN:   return M_any;
            case SYS_H_RES_FE56:    return M_fe56;
            case SYS_VAC_HARMONICS: return M_vac;
            default:                return 0.0;
        }
    }

    double get_r(int idx) const {
        switch (idx) {
            case SYS_W2_TRIADIC:    return r_w2t;
            case SYS_SGRA_BUOY:     return r_sgra_b;
            case SYS_PILLARS_DPM:   return r_pil_d;
            case SYS_RESONANCE_26D: return r_res26;
            case SYS_UTE2_SC:       return r_ute2;
            case SYS_ANYONS_CERN:   return r_any;
            case SYS_H_RES_FE56:    return r_fe56;
            case SYS_VAC_HARMONICS: return r_vac;
            default:                return 1.0;
        }
    }

    double get_DPM_res(int idx) const {
        switch (idx) {
            case SYS_W2_TRIADIC:   return DPM_res_w2;
            case SYS_SGRA_BUOY:    return DPM_res_sgra;
            case SYS_PILLARS_DPM:  return DPM_res_pil;
            default:               return 1.0;   // other systems: unscaled
        }
    }

    /// Effective magnetic field per system (for U_m computation)
    double get_B_eff(int idx) const {
        switch (idx) {
            case SYS_W2_TRIADIC:    return 1.0e-5;   // T  typical OB-wind ISM
            case SYS_SGRA_BUOY:     return 1.0e-3;   // T  Galactic Centre
            case SYS_PILLARS_DPM:   return 1.0e-5;   // T  ISM pillar
            case SYS_RESONANCE_26D: return 1.0e-5;   // T  reference
            case SYS_UTE2_SC:       return B_ute2;   // T  UTe2 lab field (16 T)
            case SYS_ANYONS_CERN:   return 1.0;       // T  experiment bench
            case SYS_H_RES_FE56:    return 30.0;      // T  hyperfine at Fe nucleus
            case SYS_VAC_HARMONICS: return 1.0e-9;    // T  IGM background
            default:                return 1.0e-5;
        }
    }

    const char* system_name(int idx) const {
        static const char* names[NUM_SYSTEMS_002] = {
            &amp;quot;Westerlund 2 [DPM Triadic]&amp;quot;,
            &amp;quot;Sgr A* [Buoyancy Dominant]&amp;quot;,
            &amp;quot;Pillars of Creation [DPM Resonance]&amp;quot;,
            &amp;quot;26D Resonance Layer Framework&amp;quot;,
            &amp;quot;UTe2 Topological Superconductor&amp;quot;,
            &amp;quot;Anyon Condensate [CERN 2025]&amp;quot;,
            &amp;quot;Fe-56 Nucleus [H_res Shell Model]&amp;quot;,
            &amp;quot;Cosmic Void [Vacuum Harmonics]&amp;quot;
```

```
245         };
246         return (idx &gt;= 0 &amp;&amp; idx &lt; NUM_SYSTEMS_002) ? names[idx] : &quot;Unknown&quot
247     }
248
249     const char* regime_label(int idx) const {
250         static const char* labels[NUM_SYSTEMS_002] = {
251             &quot;OB Star Cluster (6 pc, 30 kM\xe2\x98\x89)&quot;,
252             &quot;SMBH (1.27\xc3\x9710\xc2\xb9\xc2\xb0 m, 4\xc3\x9710\xe2\x81\xb6 M\xe2\x98\x89)&qu
253             &quot;Star-Forming Pillar (5 pc, ~10\xe2\x81\xb4 M\xe2\x98\x89)&quot;,
254             &quot;26 Orthogonal Vacuum States (Solar ref.)&quot;,
255             &quot;Condensed Matter — 1 nm crystal lattice&quot;,
256             &quot;Quasiparticle — fm scale (nuclear)&quot;,
257             &quot;Atomic Nucleus — 4.35 fm (Fe-56)&quot;,
258             &quot;Cosmic Void — 1 Mpc scale&quot;
259         };
260         return (idx &gt;= 0 &amp;&amp; idx &lt; NUM_SYSTEMS_002) ? labels[idx] : &quot;Unknown&quot
261     }
262
263     const char* unique_physics_str(int idx) const {
264         static const char* phys[NUM_SYSTEMS_002] = {
265             &quot;DPM integral F_U_Bi_i = int[LENR+DPM_mom+DPM_grav+...] dx ~2.11e208 N; &quot;
266             &quot;DPM_res=1.67e3; F_LENR=1.56e36 N; x2=-1.35e172 m (quadratic); T_sf=2 Myr.&quot;,
267
268             &quot;F_U_Bi_i~-8.31e211 N (F_LENR dominant); DPM_res=1.67e7; NSC tidal; &quot;
269             &quot;Kerr spin correction a*=0.3; QPO 20-min; GW back-reaction dOmega/dt.&quot;,
270
271             &quot;DPM_res=1.67e7 (UV-enhanced); F_U_Bi_i~2.11e212 N; triadic F_Ubi=9.79e-33 N; &quo
272             &quot;f_z,CGM~1.46e-73 ([SSq]-corrected); JWST NIRCam 2022 pillar density.&quot;,
273
274             &quot;R(t)=sum_{i=1}^{26} R_i*cos(omega_i*t); R_i=F_base*(1+SSq)*exp(-SSq*i/26); &quot
275             &quot;omega_i=2pi/T_sf*i*(1+SSq); [SSq]=0.57; 26 orthogonal vacuum quantum states.&quo
276
277             &quot;delta_{n,UTe2}=(2pi)^{n/6}*exp(-SSq*n/26)*(1+f_topo)*exp(-pi); &quot;
278             &quot;B_threshold=16 T; f_topo=0.20; Andreev STM; non-unitary TQFT; n=1..9.&quot;,
279
280             &quot;F_UBii,anyons=-F_rel*(E_any/E_LEP)*Q_wave*g*exp(-delta_c^2/(2*sigma^2)); &quot;
281             &quot;delta_c=1.686; sigma=1.0; Ising braiding E_any=1 MeV; 75% CERN 2025 alignment.&qu
282
283             &quot;H_res=A_res*sin(2pi*f_res*t)+U_dp*SC_m*k_nuc+S_shell; Z=26,A=56,N=30; &quot;
284             &quot;S_shell=0.20 (near magic 28); Fe-56 binding 8.79 MeV/nucleon (global peak).&quot
285
286             &quot;U_g2=sum H_m*(1-exp(-SSq*m))*cos(omega*t_n); H_m=sum(1/k)*f_Ub; &quot;
287             &quot;V_series=sum(1/n^26)*SSq^n~0.570=Li26(SSq); dynamic SSq(n,t).&quot;
288         };
289         return (idx &gt;= 0 &amp;&amp; idx &lt; NUM_SYSTEMS_002) ? phys[idx] : &quot;&quot;;
290     }
291
292     void get_ext(int idx, double&amp; M_ext, double&amp; r_ext) const {
293         switch (idx) {
294             case SYS_W2_TRIADIC:    M_ext = M_ext_w2t;    r_ext = r_ext_w2t;    break;
295             case SYS_SGRA_BUOY:     M_ext = M_ext_sgra_b; r_ext = r_ext_sgra_b; break;
296             case SYS_PILLARS_DPM:   M_ext = M_ext_pil_d;  r_ext = r_ext_pil_d;  break;
297             case SYS_RESONANCE_26D: M_ext = M_ext_res26;  r_ext = r_ext_res26;  break;
298             case SYS_UTE2_SC:       M_ext = M_ext_ute2;   r_ext = r_ext_ute2;   break;
299             case SYS_ANYONS_CERN:   M_ext = M_ext_any;    r_ext = r_ext_any;    break;
```

```
            case SYS_H_RES_FE56:    M_ext = M_ext_fe56;   r_ext = r_ext_fe56;   break;
            case SYS_VAC_HARMONICS: M_ext = M_ext_vac;    r_ext = r_ext_vac;    break;
            default: M_ext = M_sun; r_ext = 1.0e11; break;
        }
    }

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // Initialise all system default parameters
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    void initializeDefaults() {
        // ■■ [0] Westerlund 2 — DPM triadic ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        M_w2t      = 30000.0 * M_sun;  // 3.0e34 kg
        r_w2t      = 1.89e16;          // m  (6 pc cluster radius)
        DPM_res_w2 = 1.67e3;           // dimensionless
        F_LENR_w2  = 1.56e36;          // N
        T_sf_w2    = 6.307e13;         // s  (2 Myr)
        M_ext_w2t  = 4.0e6 * M_sun;   // SgrA* tidal (8 kpc distance)
        r_ext_w2t  = 2.469e20;         // m  (~8 kpc)

        // ■■ [1] Sgr A* — buoyancy dominant ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        M_sgra_b     = 4.0e6 * M_sun;    // 7.956e36 kg
        r_sgra_b     = 1.27e10;           // m  (~0.09 AU, G2 cloud periastron)
        DPM_res_sgra = 1.67e7;            // NSC-enhanced
        F_LENR_sgra  = 6.16e39;           // N  (F_LENR &gt;&gt; F_rel at this scale)
        T_sf_sgra    = 3.156e11;          // s  (10 kyr accretion cycle)
        M_ext_sgra_b = 1.0e11 * M_sun;   // NSC mass
        r_ext_sgra_b = 9.461e16;          // m  (~3 pc, NSC half-light radius)

        // ■■ [2] Pillars of Creation — DPM resonance ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        M_pil_d    = 10100.0 * M_sun;  // 2.009e34 kg
        r_pil_d    = 4.73e16;          // m  (5 pc pillar height)
        DPM_res_pil = 1.67e7;          // UV-ionisation enhanced
        F_LENR_pil  = 1.56e36;         // N
        T_sf_pil    = 3.156e13;        // s  (1 Myr)
        M_ext_pil_d = 40.0 * M_sun;   // Eta Carinae (η Car) + Trumpler 14
        r_ext_pil_d = 6.17e16;         // m  (~6.5 pc separation)

        // ■■ [3] 26D resonance layer ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        M_res26        = 1.0 * M_sun;    // reference: 1 M■
        r_res26        = 6.957e8;        // m  (R_sun — reference scale)
        N_layers_res26 = 26.0;           // 26 quantum vacuum states
        T_sf_res26     = 3.156e13;       // s  (1 Myr — resonance period)
        SSq_res26      = SSq;            // 0.57  (calibrated [SSq])
        M_ext_res26    = 4.0e6 * M_sun; // SgrA* (cosmological boundary)
        r_ext_res26    = 1.27e10;        // m  (ISCO scale)

        // ■■ [4] UTe2 topological superconductor ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        M_ute2      = m_e_kg;    // 9.109e-31 kg  (electron mass as reference)
        r_ute2      = 1.0e-9;   // m  (1 nm crystal unit cell)
        B_ute2      = 16.0;     // T  (superconducting threshold)
        f_topo_ute2 = 0.20;     // dimensionless (centre 0.1-0.3 range)
        M_ext_ute2  = m_p_kg;   // proton mass (nuclear context)
        r_ext_ute2  = 1.0e-10;  // m  (Bohr radius, atomic scale)

        // ■■ [5] Anyon condensate (CERN 2025) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
        M_any        = m_e_kg;         // 9.109e-31 kg
        r_any        = 1.0e-15;        // m  (1 fm nuclear interaction scale)
        E_anyons_loc = 1.602e-13;      // J  (1 MeV Ising braiding energy)
        delta_c_loc  = 1.686;          // Press-Schechter collapse threshold
        sigma_loc    = 1.0;            // non-semisimple TQFT variance
        M_ext_any    = m_p_kg;         // proton (nuclear context)
        r_ext_any    = 1.0e-15;        // m  (fm scale)

        // [redacted] [6] H_res nuclear shell — Fe-56 [redacted]
        M_fe56     = 56.0 * m_u_kg;   // 9.299e-26 kg  (56 atomic mass units)
        r_fe56     = 4.35e-15;        // m  (4.35 fm nuclear radius, r = r[redacted] A^{1/3})
        Z_fe56     = 26.0;            // atomic number
        A_fe56     = 56.0;            // mass number
        N_fe56     = 30.0;            // neutron number (A-Z)
        M_ext_fe56 = m_e_kg;          // electron mass (atomic context)
        r_ext_fe56 = 5.292e-11;       // m  (Bohr radius a[redacted])

        // [redacted] [7] Vacuum density series + buoyancy harmonics [redacted]
        M_vac          = 1.0 * M_sun;      // cosmological reference (1 M[redacted])
        r_vac          = 3.086e22;          // m  (1 Mpc Hubble volume element)
        N_harmonics_vac = 26.0;            // harmonic series order
        gamma_vac_loc  = 5.787e-10;        // s[redacted]¹  (5×10[redacted] day[redacted]¹)
        M_ext_vac      = 1.0e14 * M_sun;  // galaxy cluster (Coma scale)
        r_ext_vac      = 3.086e23;          // m  (10 Mpc)

        // Compute scalability score
        double r_min   = r_any;         // 1e-15 m  (fm, anyons)
        double r_max1  = r_vac;         // 3.086e22 m (1 Mpc)
        scalability_score = std::min(std::log10(r_max1 / r_min) / 40.0, 1.0);
    }

public:
    // [redacted]
    // Constructor / Destructor
    // [redacted]
    UQFFLearningAssessment002() {
        initializeDefaults();
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[LOG] UQFFLearningAssessment002 constructed (&quot;
                      &lt;&lt; NUM_SYSTEMS_002 &lt;&lt; &quot; systems).\n&quot;;
    }
    ~UQFFLearningAssessment002() = default;

    // [redacted]
    // §A  Core UQFF Compute Methods (mirror _001.cpp interface)
    // [redacted]

    /// PhD advancement score (0–100%).
    /// Weighted: diversity × dynamic × scalability × coverage, equal weights.
    double compute_advancement() const {
        double score = (diversity_score + dynamic_score
                        + scalability_score + coverage_score) / 4.0 * 100.0;
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[ADVANC] PhD score = &quot; &lt;&lt; score &lt;&lt; &quot; %\
        return score;
```

```
    }

    /// g_base = G·M/r² for system idx.
    double compute_g_base(int idx) const {
        return g_base_for(idx);
    }

    /// Tier-1 buoyancy: U_bi = ½ × g_base.
    double compute_Ubi(int idx) const {
        return 0.5 * g_base_for(idx);
    }

    /// Tier-2 F_UBii: standard UQFF oscillatory coupling, scaled by DPM_res.
    /// For PhD systems using specialised methods (26D, UTe2, anyons, H_res,
    /// vacuum), this provides the standardised comparison baseline.
    double compute_F_UBii(int idx, double t) const {
        double g   = g_base_for(idx);
        double M   = get_M(idx);
        double r   = get_r(idx);
        double dpm = get_DPM_res(idx);   // 1.0 for systems [3]–[7]
        return -beta_i * g * omega_g * (M / r) * U_UA
               * std::cos(UQFF_LA002_PI * t) * dpm;
    }

    /// Tier-3: external-body tidal buoyancy F_Ub_i.
    double compute_Ub_i(int idx, double t) const {
        double g = g_base_for(idx);
        double M_ext, r_ext;
        get_ext(idx, M_ext, r_ext);
        return -beta_i * g * omega_g * (M_ext / r_ext) * U_UA
               * std::cos(UQFF_LA002_PI * t);
    }

    /// Log■■ of scale range across all 8 systems (fm to Mpc).
    double compute_scale_range() const {
        return std::log10(r_vac / r_any);   // log10(3.086e22 / 1e-15) ≈ 37.5
    }

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // §B  PhD-Exclusive Compute Methods
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

    /// §B.1  log■■|F_U_Bi_i| for the three DPM-integral systems.
    /// Returns the stored calibrated logarithmic scale of the DPM integral.
    double compute_DPM_log_scale(int idx) const {
        switch (idx) {
            case SYS_W2_TRIADIC:   return 208.32;   // log10(2.11e208)
            case SYS_SGRA_BUOY:    return 211.92;   // log10(8.31e211)
            case SYS_PILLARS_DPM:  return 212.32;   // log10(2.11e212)
            default:               return std::log10(std::abs(compute_F_UBii(idx, 0.0)));
        }
    }

    /// §B.2  Full 26-layer resonance sum R(t).
    /// R(t) = Σ■■■²■ R_{Ug1,i} · cos(ω_{Ug1,i} · t)
```

```
    /// R_{Ug1,i} = g_base · (1+SSq) · exp(−SSq·i/26)
    /// ω_{Ug1,i} = (2π/T_sf) · i · (1+SSq)
    double compute_26D_resonance_sum(int idx, double t) const {
        double F_base  = g_base_for(idx);
        double T_sf    = (idx == SYS_RESONANCE_26D) ? T_sf_res26
                       : (idx == SYS_W2_TRIADIC)    ? T_sf_w2
                       : (idx == SYS_PILLARS_DPM)   ? T_sf_pil
                       :                              T_sf_res26;
        double ssq_loc = (idx == SYS_RESONANCE_26D) ? SSq_res26 : SSq;
        double sum     = 0.0;
        for (int i = 1; i &lt;= 26; ++i) {
            double R_i     = F_base * (1.0 + ssq_loc)
                           * std::exp(-ssq_loc * i / 26.0);
            double omega_i = UQFF_LA002_TWO_PI / T_sf * i * (1.0 + ssq_loc);
            sum += R_i * std::cos(omega_i * t);
        }
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[26D] R(t=&quot; &lt;&lt; t &lt;&lt; &quot;) = &quot; &lt;&lt
        return sum;
    }

    /// §B.3  UTe2 δ_n series coefficient at layer n (1-indexed).
    /// δ_{n,UTe2} = (2π)^{n/6} × exp(−[SSq]·n/26) × (1+f_topo) × exp(−π)
    double compute_delta_n_UTe2(int n) const {
        double phi_term   = std::pow(UQFF_LA002_TWO_PI, static_cast&lt;double&gt;(n) / 6.0);
        double decay_term = std::exp(-SSq * n / 26.0);
        double topo_boost = 1.0 + f_topo_ute2;
        double cosmic_exp = std::exp(-UQFF_LA002_PI);   // t_n → 0 baseline
        double result     = phi_term * decay_term * topo_boost * cosmic_exp;
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[UTe2 delta_&quot; &lt;&lt; n &lt;&lt; &quot;] = &quot; &lt;&
        return result;
    }

    /// §B.4  Anyon condensate F_UBii.
    /// F_UBii,anyons = −F_rel × (E_any/E_LEP) × Q_wave × g(r,t) × exp(−δ_c²/2σ²)
    double compute_F_UBii_anyons(double g_field = -1.0) const {
        if (g_field &lt; 0.0) g_field = g_base_for(SYS_ANYONS_CERN);
        double gauss  = std::exp(-delta_c_loc * delta_c_loc / (2.0 * sigma_loc * sigma_loc));
        double result = -F_rel * (E_anyons_loc / E_LEP) * Q_wave * g_field * gauss;
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[Anyons] F_UBii = &quot; &lt;&lt; result &lt;&lt; &quot; N (g
        return result;
    }

    /// §B.5  H_res nuclear shell amplitude (DC static approximation).
    /// Full H_res oscillates at nuclear transition frequency; here we return
    /// the time-averaged DC contribution: A_res + U_dp·SC_m·k_nuc + S_shell.
    /// Z = atomic number, A = mass number, N_n = neutron number.
    double compute_H_res(double Z, double A, double N_n) const {
        // Pairing delta: even-even → +0.1; odd-odd → -0.1; mixed → 0.0
        bool Z_even = (static_cast&lt;int&gt;(Z) % 2 == 0);
        bool N_even = (static_cast&lt;int&gt;(N_n) % 2 == 0);
        double delta_pair = (Z_even &amp;&amp; N_even) ? 0.1 : ((!Z_even) &amp;&amp; (!N_even)) ?
```

```
520         // Shell correction via proximity to magic numbers
521         constexpr int magic[7] = {2, 8, 20, 28, 50, 82, 126};
522         double S_shell = 0.0;
523         for (int m : magic) {
524             if (std::abs(static_cast&lt;int&gt;(Z) - m) &lt;= 2)   S_shell += 0.1;
525             if (std::abs(static_cast&lt;int&gt;(N_n) - m) &lt;= 2) S_shell += 0.1;
526         }
527
528         double A_res  = k_A_hres * Z * (A / A_H_ref) * (1.0 + delta_pair);
529         double k_nuc  = k0_nuc * (N_n / Z) * (1.0 + delta_pair);
530         double U_dp   = k0_nuc * (A * A_H_ref) / std::max(A * A * 0.09, 1.0e-30);
531         double SC_m   = 1.0;   // exact
532         double H_result = A_res + U_dp * SC_m * k_nuc + S_shell;
533         if (logging_enabled)
534             std::cout &lt;&lt; &quot;[H_res Z=&quot; &lt;&lt; (int)Z &lt;&lt; &quot;,A=&quot; &lt;&
535                       &lt;&lt; &quot;] A_res=&quot; &lt;&lt; A_res &lt;&lt; &quot; S_shell=&quot; &
536                       &lt;&lt; &quot; -&gt; H_res=&quot; &lt;&lt; H_result &lt;&lt; &quot;\n&quot;
537         return H_result;
538     }
539
540     /// §B.6  U_g2 buoyancy harmonics series (truncated at M_terms).
541     /// U_g2 = Σ_{m=1}^{M} H_m × (1 − e^{−[SSq]m}) × cos(ω_{Ug2} × t_n)
542     /// H_m  = Σ_{k=1}^m (1/k) × f_Ub
543     /// t_n  = t_seconds / t_Hubble
544     double compute_Ug2_harmonics(double t_seconds, int M_terms = 26) const {
545         double t_n       = t_seconds / t_Hubble;
546         double omega_ug2 = UQFF_LA002_TWO_PI / t_Hubble;   // 1.44e-17 rad/s
547         double f_ub      = f_Ub_cal;   // 2.20e7 (calibrated)
548         double result    = 0.0;
549         double H_m       = 0.0;
550         for (int m = 1; m &lt;= M_terms; ++m) {
551             H_m    += f_ub / static_cast&lt;double&gt;(m);
552             double damp = 1.0 - std::exp(-SSq * m);
553             result += H_m * damp * std::cos(omega_ug2 * t_n);
554         }
555         if (logging_enabled)
556             std::cout &lt;&lt; &quot;[Ug2 harmonics, M=&quot; &lt;&lt; M_terms
557                       &lt;&lt; &quot;, t_n=&quot; &lt;&lt; t_n &lt;&lt; &quot;] = &quot; &lt;&lt; r
558         return result;
559     }
560
561     /// §B.7  Vacuum Density Series (partial sum to N_terms).
562     /// V_series = Σ_{n=1}^{N} (1/n²■) × [SSq]^n  →  Li■■([SSq]) ≈ 0.570
563     double compute_Vacuum_Series(int N_terms = 26) const {
564         double result = 0.0;
565         double ssq_n  = SSq;
566         for (int n = 1; n &lt;= N_terms; ++n) {
567             result += ssq_n / std::pow(static_cast&lt;double&gt;(n), 26.0);
568             ssq_n  *= SSq;
569         }
570         if (logging_enabled)
571             std::cout &lt;&lt; &quot;[VacSeries N=&quot; &lt;&lt; N_terms &lt;&lt; &quot;] = &quot
572         return result;
573     }
574
```

```
    /// §B.8  Full U_m (single-j approximation with Heaviside correction).
    /// U_m = (μ_j/r_j) × (1 − e^{−γt}·cos(πt_n)) × P_SCm × E_react
    ///       × (1 + 10¹³·f_H) × (1+f_quasi) × e^{−[SSq]}
    double compute_Um_full(int idx, double t) const {
        double M    = get_M(idx);
        double r    = get_r(idx);
        double B_eff = get_B_eff(idx);
        double t_n  = t / t_Hubble;

        // Calibrated μ_j: Bohr magneton scale, field-enhanced
        double mu_j = mu_B_SI * B_eff / std::max(hbar * 1.0e-12, 1.0e-50);
        double gamma_eff = (idx == SYS_VAC_HARMONICS) ? gamma_vac_loc : gamma_d;

        double exp_term  = 1.0 - std::exp(-gamma_eff * t) * std::cos(UQFF_LA002_PI * t_n);
        double P_SCm     = 1.0;
        double E_react   = U_UA;    // reaction energy proxy
        double f_Heaviside = 0.01;
        double f_quasi   = 0.01;

        double Um_val = (mu_j / r) * exp_term * P_SCm * E_react
                        * (1.0 + 1.0e13 * f_Heaviside) * (1.0 + f_quasi)
                        * std::exp(-SSq);
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[Um sys[&quot; &lt;&lt; idx &lt;&lt; &quot;] t=&quot; &lt;&lt
        return Um_val;
    }

    /// §B.9  Dynamic [SSq](n, t_seconds): vacuum entanglement entropy at layer n.
    /// [SSq](n,t) = log(ρ_SCm/ρ_UA) × n × e^{−(π−t_n)}
    double compute_SSq_dynamic(int n, double t_seconds) const {
        double t_n      = t_seconds / t_Hubble;
        double log_ratio = std::log(rho_vac_SCm / rho_vac_UA);   // ≈ -2.303
        return log_ratio * n * std::exp(-(UQFF_LA002_PI - t_n));
    }

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // §C  Print / Display Methods
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

    /// Print the PhD-level student guide.
    void printPhDGuide() const {
        std::cout &lt;&lt; &quot;\n&quot;;
        std::cout &lt;&lt; &quot;==================================================================
        std::cout &lt;&lt; &quot;  UQFF Learning Assessment 002 — PhD Research Edition (v1.0)\n&quo
        std::cout &lt;&lt; &quot;  Audience: PhD quantum physics / astrophysics / cosmology\n&quot
        std::cout &lt;&lt; &quot;==================================================================

        std::cout &lt;&lt; &quot;FRAMEWORK OVERVIEW\n&quot;;
        std::cout &lt;&lt; &quot;------------------\n&quot;;
        std::cout &lt;&lt; &quot;  This assessment extends the Research Student Edition (_001)\n&q
        std::cout &lt;&lt; &quot;  to 8 systems spanning 37+ log-decades of scale (1 fm -&gt; 1 Mp
        std::cout &lt;&lt; &quot;  10 unique PhD-level physics processes are evaluated:\n\n&quot;;
        std::cout &lt;&lt; &quot;    1. DPM 4-component buoyancy integral (F_U_Bi_i ~ 10^208..211 
        std::cout &lt;&lt; &quot;    2. 26D resonance sum R(t) = sum_{i=1}^{26} R_i cos(omega_i t)
        std::cout &lt;&lt; &quot;    3. UTe2 topological SC delta_n series (n=1..9, f_topo=0.20)\n
```

```
        std::cout &lt;&lt; &quot;    4. Anyon F_UBii: Gaussian collapse exp(-delta_c^2/2sigma^2)\n&
        std::cout &lt;&lt; &quot;    5. H_res nuclear shell model (Fe-56 peak, magic numbers)\n&quo
        std::cout &lt;&lt; &quot;    6. Vacuum Density Series: sum(1/n^26)[SSq]^n = Li26([SSq])\n&
        std::cout &lt;&lt; &quot;    7. Buoyancy Harmonics: U_g2 = sum H_m (1-e^{-[SSq]m}) cos(...
        std::cout &lt;&lt; &quot;    8. Full Um: P_SCm x E_react x f_Heaviside x f_quasi x e^{-SSq
        std::cout &lt;&lt; &quot;    9. Dynamic [SSq](n,t) = log(rho_SCm/rho_UA) x n x e^{-(pi-t_n
        std::cout &lt;&lt; &quot;   10. F_rel / E_LEP / Q_wave relativistic calibration (2024 LEP)\
        std::cout &lt;&lt; &quot;SYSTEM CATALOGUE\n&quot;;
        std::cout &lt;&lt; &quot;----------------\n&quot;;
        std::cout &lt;&lt; std::left &lt;&lt; std::setw(4)  &lt;&lt; &quot;Idx&quot;
                  &lt;&lt; std::setw(42) &lt;&lt; &quot;Name&quot;
                  &lt;&lt; std::setw(20) &lt;&lt; &quot;g_base (m/s^2)&quot;
                  &lt;&lt; std::setw(24) &lt;&lt; &quot;Primary PhD Method\n&quot;;
        std::cout &lt;&lt; std::string(90, &#39;-&#39;) &lt;&lt; &quot;\n&quot;;
        const char* methods[NUM_SYSTEMS_002] = {
            &quot;DPM integral log10|F|&quot;, &quot;DPM integral log10|F|&quot;, &quot;DPM integra
            &quot;26D resonance sum R(t)&quot;, &quot;UTe2 delta_n series&quot;,   &quot;Anyon F_U
            &quot;H_res(Z,A,N)&quot;, &quot;Ug2 harmonics + VacSeries&quot;
        };
        for (int i = 0; i &lt; NUM_SYSTEMS_002; ++i) {
            std::cout &lt;&lt; std::left
                      &lt;&lt; std::setw(4)  &lt;&lt; i
                      &lt;&lt; std::setw(42) &lt;&lt; system_name(i)
                      &lt;&lt; std::setw(20) &lt;&lt; std::scientific &lt;&lt; std::setprecision(3
                      &lt;&lt; compute_g_base(i)
                      &lt;&lt; std::setw(24) &lt;&lt; methods[i] &lt;&lt; &quot;\n&quot;;
        }

        std::cout &lt;&lt; &quot;\nSCALE RANGE\n&quot;;
        std::cout &lt;&lt; &quot;-----------\n&quot;;
        std::cout &lt;&lt; &quot;  r_min = r_any  = &quot; &lt;&lt; r_any
                  &lt;&lt; &quot; m  [sys 5: anyon, nuclear scale]\n&quot;;
        std::cout &lt;&lt; &quot;  r_max = r_vac  = &quot; &lt;&lt; r_vac
                  &lt;&lt; &quot; m  [sys 7: 1 Mpc cosmic void]\n&quot;;
        std::cout &lt;&lt; &quot;  log10(r_max/r_min) = &quot; &lt;&lt; std::fixed &lt;&lt; std::se
                  &lt;&lt; compute_scale_range() &lt;&lt; &quot; decades  (37.49 expected)\n&quot;;

        std::cout &lt;&lt; &quot;\nKEY PhD CONSTANTS (UQFF PhD Edition)\n&quot;;
        std::cout &lt;&lt; &quot;-------------------------------------\n&quot;;
        std::cout &lt;&lt; &quot;  F_rel       = &quot; &lt;&lt; std::scientific &lt;&lt; F_rel
                  &lt;&lt; &quot; N   (2024 LEP calibration)\n&quot;;
        std::cout &lt;&lt; &quot;  E_LEP       = &quot; &lt;&lt; E_LEP &lt;&lt; &quot; J   (200 G
        std::cout &lt;&lt; &quot;  Q_wave      = &quot; &lt;&lt; Q_wave &lt;&lt; &quot;    (coupli
        std::cout &lt;&lt; &quot;  [SSq]       = &quot; &lt;&lt; SSq    &lt;&lt; &quot;    (vacuum
        std::cout &lt;&lt; &quot;  gamma_decay  = &quot; &lt;&lt; gamma_d &lt;&lt; &quot; s^-1  (5
        std::cout &lt;&lt; &quot;  delta_c_any  = &quot; &lt;&lt; delta_c_any &lt;&lt; &quot;    (
        std::cout &lt;&lt; &quot;  B_thr(UTe2)  = &quot; &lt;&lt; 16.0   &lt;&lt; &quot; T\n&quot;;
        std::cout &lt;&lt; &quot;  Li26([SSq])  = &quot; &lt;&lt; compute_Vacuum_Series(100) &lt;&
        std::cout &lt;&lt; &quot;\nUTe2 delta_n SERIES (n=1..9)\n&quot;;
        std::cout &lt;&lt; &quot;------------------------------\n&quot;;
        for (int n = 1; n &lt;= 9; ++n)
            std::cout &lt;&lt; &quot;  delta_&quot; &lt;&lt; n &lt;&lt; &quot; = &quot; &lt;&lt; s
                      &lt;&lt; std::setprecision(4) &lt;&lt; compute_delta_n_UTe2(n) &lt;&lt; &quo
```

```
685
686         std::cout &lt;&lt; &quot;\nH_res HIGHLIGHTS\n&quot;;
687         std::cout &lt;&lt; &quot;----------------\n&quot;;
688         std::cout &lt;&lt; &quot;  Fe-56  (Z=26, A=56, N=30): H_res = &quot; &lt;&lt; std::fixed
689                   &lt;&lt; std::setprecision(4) &lt;&lt; compute_H_res(Z_fe56, A_fe56, N_fe56)
690                   &lt;&lt; &quot;  [peak binding 8.79 MeV/nucleon]\n&quot;;
691         std::cout &lt;&lt; &quot;  O-16   (Z=8,  A=16, N=8):  H_res = &quot;
692                   &lt;&lt; compute_H_res(8,16,8) &lt;&lt; &quot;  [doubly-magic]\n&quot;;
693         std::cout &lt;&lt; &quot;  Pb-208 (Z=82, A=208,N=126): H_res = &quot;
694                   &lt;&lt; compute_H_res(82,208,126) &lt;&lt; &quot;  [doubly-magic, terminus]\n&qu
695         std::cout &lt;&lt; &quot;  H-1    (Z=1,  A=1,  N=0):  H_res = &quot;
696                   &lt;&lt; compute_H_res(1,1,0) &lt;&lt; &quot;  [lightest, reference A_H=1]\n&quot
697
698         std::cout &lt;&lt; &quot;\nDPM INTEGRAL LOG-SCALE RESULTS\n&quot;;
699         std::cout &lt;&lt; &quot;--------------------------------\n&quot;;
700         for (int i = 0; i &lt;= 2; ++i)
701             std::cout &lt;&lt; &quot;  sys[&quot; &lt;&lt; i &lt;&lt; &quot;] &quot; &lt;&lt; std:
702                       &lt;&lt; system_name(i) &lt;&lt; &quot;: log10|F_U_Bi_i| = &quot;
703                       &lt;&lt; std::fixed &lt;&lt; std::setprecision(2)
704                       &lt;&lt; compute_DPM_log_scale(i) &lt;&lt; &quot;\n&quot;;
705
706         std::cout &lt;&lt; &quot;\nADVANCEMENT METRIC (PhD)\n&quot;;
707         std::cout &lt;&lt; &quot;-------------------------\n&quot;;
708         std::cout &lt;&lt; &quot;  diversity    = &quot; &lt;&lt; std::fixed &lt;&lt; std::setprec
709                   &lt;&lt; diversity_score * 100.0 &lt;&lt; &quot; %  (8/8 regimes)\n&quot;;
710         std::cout &lt;&lt; &quot;  dynamic      = &quot; &lt;&lt; dynamic_score * 100.0
711                   &lt;&lt; &quot; %  (10/10 PhD processes)\n&quot;;
712         std::cout &lt;&lt; &quot;  scalability  = &quot; &lt;&lt; scalability_score * 100.0
713                   &lt;&lt; &quot; %  (&quot; &lt;&lt; compute_scale_range() &lt;&lt; &quot; / 40 d
714         std::cout &lt;&lt; &quot;  coverage     = &quot; &lt;&lt; coverage_score * 100.0
715                   &lt;&lt; &quot; %  (8/8 systems)\n&quot;;
716         std::cout &lt;&lt; &quot;  TOTAL        = &quot; &lt;&lt; compute_advancement() &lt;&lt; &
717     }
718
719     /// Print sorted comparative analysis table at time t (seconds).
720     void printComparativeAnalysis(double t = 0.0) const {
721         std::cout &lt;&lt; &quot;\n&quot;;
722         std::cout &lt;&lt; &quot;=========================================================================
723         std::cout &lt;&lt; &quot;  Comparative UQFF Buoyancy Analysis  (t = &quot;
724                   &lt;&lt; std::scientific &lt;&lt; t &lt;&lt; &quot; s)\n&quot;;
725         std::cout &lt;&lt; &quot;=========================================================================
726         std::cout &lt;&lt; std::left
727                   &lt;&lt; std::setw(4)  &lt;&lt; &quot;Idx&quot;
728                   &lt;&lt; std::setw(34) &lt;&lt; &quot;System&quot;
729                   &lt;&lt; std::setw(16) &lt;&lt; &quot;g_base (m/s^2)&quot;
730                   &lt;&lt; std::setw(16) &lt;&lt; &quot;U_bi (m/s^2)&quot;
731                   &lt;&lt; std::setw(18) &lt;&lt; &quot;F_UBii (N,scaled)&quot;
732                   &lt;&lt; std::setw(18) &lt;&lt; &quot;U_m (full)\n&quot;;
733         std::cout &lt;&lt; std::string(106, &#39;-&#39;) &lt;&lt; &quot;\n&quot;;
734
735         // Collect sorted order by g_base
736         std::array&lt;int, NUM_SYSTEMS_002&gt; order;
737         std::iota(order.begin(), order.end(), 0);
738         std::sort(order.begin(), order.end(), [this](int a, int b) {
739             return std::abs(compute_g_base(a)) &gt; std::abs(compute_g_base(b));
```

```
740         });
741
742         for (int i : order) {
743             double g    = compute_g_base(i);
744             double ubi  = compute_Ubi(i);
745             double fii  = compute_F_UBii(i, t);
746             double um   = compute_Um_full(i, t);
747             std::cout &lt;&lt; std::left
748                       &lt;&lt; std::setw(4)  &lt;&lt; i
749                       &lt;&lt; std::setw(34) &lt;&lt; system_name(i)
750                       &lt;&lt; std::setw(16) &lt;&lt; std::scientific &lt;&lt; std::setprecision(3
751                       &lt;&lt; std::setw(16) &lt;&lt; ubi
752                       &lt;&lt; std::setw(18) &lt;&lt; fii
753                       &lt;&lt; std::setw(18) &lt;&lt; um &lt;&lt; &quot;\n&quot;;
754         }
755         std::cout &lt;&lt; &quot;\n&quot;;
756         std::cout &lt;&lt; &quot;  NOTE: F_UBii shown is Tier-2 standard (DPM-scaled for sys[0-2])
757         std::cout &lt;&lt; &quot;  For calibrated DPM integral use compute_DPM_log_scale(idx).\n&qu
758         std::cout &lt;&lt; &quot;  For anyon: compute_F_UBii_anyons(); 26D: compute_26D_resonance_s
759     }
760
761     /// Print compact parameter dump for all 8 systems.
762     void printParameters() const {
763         std::cout &lt;&lt; &quot;\n--- UQFFLearningAssessment002 Parameters ---\n&quot;;
764         std::cout &lt;&lt; std::scientific &lt;&lt; std::setprecision(4);
765         auto row = [&amp;](const char* tag, double v) {
766             std::cout &lt;&lt; &quot;  &quot; &lt;&lt; std::left &lt;&lt; std::setw(22) &lt;&lt; ta
767         };
768         std::cout &lt;&lt; &quot;[0] Westerlund 2 Triadic\n&quot;;
769         row(&quot;M_w2t&quot;,       M_w2t);       row(&quot;r_w2t&quot;,     r_w2t);
770         row(&quot;DPM_res_w2&quot;,  DPM_res_w2);  row(&quot;F_LENR_w2&quot;, F_LENR_w2);
771         row(&quot;T_sf_w2&quot;,     T_sf_w2);
772         std::cout &lt;&lt; &quot;[1] Sgr A* Buoyancy\n&quot;;
773         row(&quot;M_sgra_b&quot;,     M_sgra_b);   row(&quot;r_sgra_b&quot;,   r_sgra_b);
774         row(&quot;DPM_res_sgra&quot;, DPM_res_sgra); row(&quot;F_LENR_sgra&quot;, F_LENR_sgra);
775         std::cout &lt;&lt; &quot;[2] Pillars DPM\n&quot;;
776         row(&quot;M_pil_d&quot;,     M_pil_d);     row(&quot;r_pil_d&quot;,   r_pil_d);
777         row(&quot;DPM_res_pil&quot;, DPM_res_pil); row(&quot;F_LENR_pil&quot;, F_LENR_pil);
778         std::cout &lt;&lt; &quot;[3] 26D Resonance\n&quot;;
779         row(&quot;M_res26&quot;,     M_res26);     row(&quot;r_res26&quot;,    r_res26);
780         row(&quot;N_layers&quot;,    N_layers_res26); row(&quot;T_sf_res26&quot;, T_sf_res26);
781         row(&quot;SSq_res26&quot;,   SSq_res26);
782         std::cout &lt;&lt; &quot;[4] UTe2 SC\n&quot;;
783         row(&quot;M_ute2&quot;,      M_ute2);      row(&quot;r_ute2&quot;,    r_ute2);
784         row(&quot;B_ute2&quot;,      B_ute2);      row(&quot;f_topo_ute2&quot;, f_topo_ute2);
785         std::cout &lt;&lt; &quot;[5] Anyons CERN\n&quot;;
786         row(&quot;M_any&quot;,       M_any);       row(&quot;r_any&quot;,     r_any);
787         row(&quot;E_anyons_loc&quot;, E_anyons_loc); row(&quot;delta_c_loc&quot;, delta_c_loc);
788         std::cout &lt;&lt; &quot;[6] H_res Fe-56\n&quot;;
789         row(&quot;M_fe56&quot;,      M_fe56);      row(&quot;r_fe56&quot;,    r_fe56);
790         row(&quot;Z_fe56&quot;,      Z_fe56);      row(&quot;A_fe56&quot;,    A_fe56);
791         row(&quot;N_fe56&quot;,      N_fe56);
792         std::cout &lt;&lt; &quot;[7] Vacuum Harmonics\n&quot;;
793         row(&quot;M_vac&quot;,       M_vac);       row(&quot;r_vac&quot;,     r_vac);
794         row(&quot;N_harmonics&quot;, N_harmonics_vac); row(&quot;gamma_vac&quot;, gamma_vac_loc);
```

```
        std::cout &lt;&lt; &quot;---\n&quot;;
    }

    /// Run a representative PhD-level research state example.
    /// Evaluates all 8 systems at a 1-Myr reference time.
    void exampleAtResearchState() const {
        constexpr double t_1Myr = 3.156e13;   // s  (1 million years)
        printPhDGuide();
        printComparativeAnalysis(t_1Myr);

        std::cout &lt;&lt; &quot;\nPHD SPECIFIC RESULTS AT t = 1 Myr\n&quot;;
        std::cout &lt;&lt; &quot;-----------------------------------\n&quot;;

        // 26D resonance sum
        double R26 = compute_26D_resonance_sum(SYS_RESONANCE_26D, t_1Myr);
        std::cout &lt;&lt; &quot;  R(t=1Myr, 26D)         = &quot; &lt;&lt; std::scientific
                  &lt;&lt; std::setprecision(4) &lt;&lt; R26 &lt;&lt; &quot; N\n&quot;;

        // Anyon F_UBii
        double F_any = compute_F_UBii_anyons();
        std::cout &lt;&lt; &quot;  F_UBii,anyons (g_any)  = &quot; &lt;&lt; F_any &lt;&lt; &quot; N

        // Vacuum series
        double V_ser = compute_Vacuum_Series(26);
        std::cout &lt;&lt; &quot;  Vac.Series(N=26)       = &quot; &lt;&lt; V_ser
                  &lt;&lt; &quot;  [Li26(0.57) approx]\n&quot;;

        // U_g2 harmonics at t=0 (cosmic reference)
        double Ug2  = compute_Ug2_harmonics(0.0, 26);
        std::cout &lt;&lt; &quot;  U_g2 harmonics(t=0)    = &quot; &lt;&lt; Ug2 &lt;&lt; &quot; [n

        // Dynamic [SSq] at n=13, t=0
        double ssq_dyn = compute_SSq_dynamic(13, t_1Myr);
        std::cout &lt;&lt; &quot;  SSq_dynamic(n=13,1Myr) = &quot; &lt;&lt; ssq_dyn &lt;&lt; &quot;

        // Full Um for Sgr A* at 1 Myr
        double Um_sgra = compute_Um_full(SYS_SGRA_BUOY, t_1Myr);
        std::cout &lt;&lt; &quot;  Um_full(SgrA*, 1Myr)   = &quot; &lt;&lt; Um_sgra &lt;&lt; &quot;

        std::cout &lt;&lt; &quot;\nDPM LOG-SCALE SUMMARY\n&quot;;
        std::cout &lt;&lt; &quot;---------------------\n&quot;;
        const char* sys_names[3] = {&quot;W2 Triadic&quot;, &quot;SgrA* Buoy&quot;, &quot;Pillars 
        for (int i = 0; i &lt; 3; ++i)
            std::cout &lt;&lt; &quot;  log10|F_U_Bi_i| (&quot; &lt;&lt; sys_names[i] &lt;&lt; &quo
                      &lt;&lt; std::fixed &lt;&lt; std::setprecision(1)
                      &lt;&lt; compute_DPM_log_scale(i) &lt;&lt; &quot;\n&quot;;

        std::cout &lt;&lt; &quot;\n  PhD Advancement Score = &quot;
                  &lt;&lt; std::fixed &lt;&lt; std::setprecision(2) &lt;&lt; compute_advancement()
                  &lt;&lt; &quot; %\n\n&quot;;
    }

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // §D  UQFF 2.0 Self-Expanding Framework — Variable Access
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```

    bool setVariable(const std::string&amp; varName, double newValue) {
        // ■■ [0] W2 Triadic ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        if      (varName == &quot;M_w2t&quot;)       M_w2t       = newValue;
        else if (varName == &quot;r_w2t&quot;)       r_w2t       = newValue;
        else if (varName == &quot;DPM_res_w2&quot;)  DPM_res_w2  = newValue;
        else if (varName == &quot;F_LENR_w2&quot;)   F_LENR_w2   = newValue;
        else if (varName == &quot;T_sf_w2&quot;)     T_sf_w2     = newValue;
        else if (varName == &quot;M_ext_w2t&quot;)   M_ext_w2t   = newValue;
        else if (varName == &quot;r_ext_w2t&quot;)   r_ext_w2t   = newValue;
        // ■■ [1] SgrA* Buoyancy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        else if (varName == &quot;M_sgra_b&quot;)    M_sgra_b    = newValue;
        else if (varName == &quot;r_sgra_b&quot;)    r_sgra_b    = newValue;
        else if (varName == &quot;DPM_res_sgra&quot;) DPM_res_sgra = newValue;
        else if (varName == &quot;F_LENR_sgra&quot;)  F_LENR_sgra  = newValue;
        else if (varName == &quot;T_sf_sgra&quot;)   T_sf_sgra   = newValue;
        else if (varName == &quot;M_ext_sgra_b&quot;) M_ext_sgra_b = newValue;
        else if (varName == &quot;r_ext_sgra_b&quot;) r_ext_sgra_b = newValue;
        // ■■ [2] Pillars DPM ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        else if (varName == &quot;M_pil_d&quot;)     M_pil_d     = newValue;
        else if (varName == &quot;r_pil_d&quot;)     r_pil_d     = newValue;
        else if (varName == &quot;DPM_res_pil&quot;)  DPM_res_pil  = newValue;
        else if (varName == &quot;F_LENR_pil&quot;)   F_LENR_pil   = newValue;
        else if (varName == &quot;T_sf_pil&quot;)     T_sf_pil     = newValue;
        else if (varName == &quot;M_ext_pil_d&quot;)  M_ext_pil_d  = newValue;
        else if (varName == &quot;r_ext_pil_d&quot;)  r_ext_pil_d  = newValue;
        // ■■ [3] 26D Resonance ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        else if (varName == &quot;M_res26&quot;)        M_res26        = newValue;
        else if (varName == &quot;r_res26&quot;)        r_res26        = newValue;
        else if (varName == &quot;N_layers_res26&quot;) N_layers_res26  = newValue;
        else if (varName == &quot;T_sf_res26&quot;)     T_sf_res26     = newValue;
        else if (varName == &quot;SSq_res26&quot;)      SSq_res26      = newValue;
        else if (varName == &quot;M_ext_res26&quot;)    M_ext_res26    = newValue;
        else if (varName == &quot;r_ext_res26&quot;)    r_ext_res26    = newValue;
        // ■■ [4] UTe2 SC ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        else if (varName == &quot;M_ute2&quot;)      M_ute2      = newValue;
        else if (varName == &quot;r_ute2&quot;)      r_ute2      = newValue;
        else if (varName == &quot;B_ute2&quot;)      B_ute2      = newValue;
        else if (varName == &quot;f_topo_ute2&quot;)  f_topo_ute2  = newValue;
        else if (varName == &quot;M_ext_ute2&quot;)   M_ext_ute2   = newValue;
        else if (varName == &quot;r_ext_ute2&quot;)   r_ext_ute2   = newValue;
        // ■■ [5] Anyons ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        else if (varName == &quot;M_any&quot;)         M_any         = newValue;
        else if (varName == &quot;r_any&quot;)         r_any         = newValue;
        else if (varName == &quot;E_anyons_loc&quot;)  E_anyons_loc  = newValue;
        else if (varName == &quot;delta_c_loc&quot;)   delta_c_loc   = newValue;
        else if (varName == &quot;sigma_loc&quot;)     sigma_loc     = newValue;
        else if (varName == &quot;M_ext_any&quot;)     M_ext_any     = newValue;
        else if (varName == &quot;r_ext_any&quot;)     r_ext_any     = newValue;
        // ■■ [6] H_res Fe-56 ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        else if (varName == &quot;M_fe56&quot;)     M_fe56     = newValue;
        else if (varName == &quot;r_fe56&quot;)     r_fe56     = newValue;
        else if (varName == &quot;Z_fe56&quot;)     Z_fe56     = newValue;
        else if (varName == &quot;A_fe56&quot;)     A_fe56     = newValue;
        else if (varName == &quot;N_fe56&quot;)     N_fe56     = newValue;
```

```
        else if (varName == &quot;M_ext_fe56&quot;) M_ext_fe56 = newValue;
        else if (varName == &quot;r_ext_fe56&quot;) r_ext_fe56 = newValue;
        // ■■ [7] Vacuum Harmonics ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        else if (varName == &quot;M_vac&quot;)           M_vac           = newValue;
        else if (varName == &quot;r_vac&quot;)           r_vac           = newValue;
        else if (varName == &quot;N_harmonics_vac&quot;) N_harmonics_vac = newValue;
        else if (varName == &quot;gamma_vac_loc&quot;)   gamma_vac_loc   = newValue;
        else if (varName == &quot;M_ext_vac&quot;)       M_ext_vac       = newValue;
        else if (varName == &quot;r_ext_vac&quot;)       r_ext_vac       = newValue;
        // ■■ Assessment metrics ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        else if (varName == &quot;diversity_score&quot;)   diversity_score   = newValue;
        else if (varName == &quot;dynamic_score&quot;)     dynamic_score     = newValue;
        else if (varName == &quot;scalability_score&quot;) scalability_score = newValue;
        else if (varName == &quot;coverage_score&quot;)    coverage_score    = newValue;
        else {
            std::cerr &lt;&lt; &quot;Error: Unknown variable &#39;&quot; &lt;&lt; varName &lt;&lt;
            return false;
        }
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[LOG] setVariable: &quot; &lt;&lt; varName &lt;&lt; &quot; = &
        return true;
    }

    bool addToVariable(const std::string&amp; varName, double delta) {
        return setVariable(varName, getVariable(varName) + delta);
    }

    bool subtractFromVariable(const std::string&amp; varName, double delta) {
        return addToVariable(varName, -delta);
    }

    double getVariable(const std::string&amp; varName) const {
        if      (varName == &quot;M_w2t&quot;)        return M_w2t;
        else if (varName == &quot;r_w2t&quot;)        return r_w2t;
        else if (varName == &quot;DPM_res_w2&quot;)   return DPM_res_w2;
        else if (varName == &quot;F_LENR_w2&quot;)    return F_LENR_w2;
        else if (varName == &quot;T_sf_w2&quot;)      return T_sf_w2;
        else if (varName == &quot;M_ext_w2t&quot;)    return M_ext_w2t;
        else if (varName == &quot;r_ext_w2t&quot;)    return r_ext_w2t;
        else if (varName == &quot;M_sgra_b&quot;)     return M_sgra_b;
        else if (varName == &quot;r_sgra_b&quot;)     return r_sgra_b;
        else if (varName == &quot;DPM_res_sgra&quot;) return DPM_res_sgra;
        else if (varName == &quot;F_LENR_sgra&quot;)  return F_LENR_sgra;
        else if (varName == &quot;T_sf_sgra&quot;)    return T_sf_sgra;
        else if (varName == &quot;M_ext_sgra_b&quot;) return M_ext_sgra_b;
        else if (varName == &quot;r_ext_sgra_b&quot;) return r_ext_sgra_b;
        else if (varName == &quot;M_pil_d&quot;)      return M_pil_d;
        else if (varName == &quot;r_pil_d&quot;)      return r_pil_d;
        else if (varName == &quot;DPM_res_pil&quot;)  return DPM_res_pil;
        else if (varName == &quot;F_LENR_pil&quot;)   return F_LENR_pil;
        else if (varName == &quot;T_sf_pil&quot;)     return T_sf_pil;
        else if (varName == &quot;M_ext_pil_d&quot;)  return M_ext_pil_d;
        else if (varName == &quot;r_ext_pil_d&quot;)  return r_ext_pil_d;
        else if (varName == &quot;M_res26&quot;)         return M_res26;
        else if (varName == &quot;r_res26&quot;)         return r_res26;
```

```cpp
        else if (varName == &quot;N_layers_res26&quot;)  return N_layers_res26;
        else if (varName == &quot;T_sf_res26&quot;)      return T_sf_res26;
        else if (varName == &quot;SSq_res26&quot;)       return SSq_res26;
        else if (varName == &quot;M_ext_res26&quot;)     return M_ext_res26;
        else if (varName == &quot;r_ext_res26&quot;)     return r_ext_res26;
        else if (varName == &quot;M_ute2&quot;)       return M_ute2;
        else if (varName == &quot;r_ute2&quot;)       return r_ute2;
        else if (varName == &quot;B_ute2&quot;)       return B_ute2;
        else if (varName == &quot;f_topo_ute2&quot;)  return f_topo_ute2;
        else if (varName == &quot;M_ext_ute2&quot;)   return M_ext_ute2;
        else if (varName == &quot;r_ext_ute2&quot;)   return r_ext_ute2;
        else if (varName == &quot;M_any&quot;)         return M_any;
        else if (varName == &quot;r_any&quot;)         return r_any;
        else if (varName == &quot;E_anyons_loc&quot;)  return E_anyons_loc;
        else if (varName == &quot;delta_c_loc&quot;)   return delta_c_loc;
        else if (varName == &quot;sigma_loc&quot;)     return sigma_loc;
        else if (varName == &quot;M_ext_any&quot;)     return M_ext_any;
        else if (varName == &quot;r_ext_any&quot;)     return r_ext_any;
        else if (varName == &quot;M_fe56&quot;)     return M_fe56;
        else if (varName == &quot;r_fe56&quot;)     return r_fe56;
        else if (varName == &quot;Z_fe56&quot;)     return Z_fe56;
        else if (varName == &quot;A_fe56&quot;)     return A_fe56;
        else if (varName == &quot;N_fe56&quot;)     return N_fe56;
        else if (varName == &quot;M_ext_fe56&quot;) return M_ext_fe56;
        else if (varName == &quot;r_ext_fe56&quot;) return r_ext_fe56;
        else if (varName == &quot;M_vac&quot;)           return M_vac;
        else if (varName == &quot;r_vac&quot;)           return r_vac;
        else if (varName == &quot;N_harmonics_vac&quot;) return N_harmonics_vac;
        else if (varName == &quot;gamma_vac_loc&quot;)   return gamma_vac_loc;
        else if (varName == &quot;M_ext_vac&quot;)       return M_ext_vac;
        else if (varName == &quot;r_ext_vac&quot;)       return r_ext_vac;

        else if (varName == &quot;diversity_score&quot;)   return diversity_score;
        else if (varName == &quot;dynamic_score&quot;)     return dynamic_score;
        else if (varName == &quot;scalability_score&quot;) return scalability_score;
        else if (varName == &quot;coverage_score&quot;)    return coverage_score;
        else {
            std::cerr &lt;&lt; &quot;Error: Unknown variable &#39;&quot; &lt;&lt; varName &lt;&lt;
            return 0.0;
        }
    }

    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    // §E  UQFF 2.0 Self-Expanding Framework — Dynamic Params &amp; State
    // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

    void setEnableLogging(bool enabled) { logging_enabled = enabled; }
    bool getLoggingEnabled() const      { return logging_enabled; }

    void setDynamicParameter(const std::string&amp; key, double value) {
        dynamic_params[key] = value;
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[LOG] Dynamic param set: &quot; &lt;&lt; key &lt;&lt; &quot; 
    }

    double getDynamicParameter(const std::string&amp; key) const {
```

```
1015          auto it = dynamic_params.find(key);
1016          if (it != dynamic_params.end()) return it-&gt;second;
1017          if (logging_enabled)
1018              std::cout &lt;&lt; &quot;[LOG] Dynamic param not found: &quot; &lt;&lt; key &lt;&lt; &
1019          return std::numeric_limits&lt;double&gt;::quiet_NaN();
1020      }
1021
1022      void exportState(const std::string&amp; filename = &quot;UQFFLearningAssessment002_state.txt&qu
1023          std::ofstream ofs(filename);
1024          if (!ofs) {
1025              std::cerr &lt;&lt; &quot;Error: Cannot open &#39;&quot; &lt;&lt; filename &lt;&lt; &quo
1026              return;
1027          }
1028          ofs &lt;&lt; &quot;# UQFFLearningAssessment002 (PhD Research Edition) state export\n&quot;
1029          ofs &lt;&lt; std::scientific &lt;&lt; std::setprecision(10);
1030          const char* names[] = {
1031              // [0] W2
1032              &quot;M_w2t&quot;, &quot;r_w2t&quot;, &quot;DPM_res_w2&quot;, &quot;F_LENR_w2&quot;, &
1033              &quot;M_ext_w2t&quot;, &quot;r_ext_w2t&quot;,
1034              // [1] SgrA*
1035              &quot;M_sgra_b&quot;, &quot;r_sgra_b&quot;, &quot;DPM_res_sgra&quot;, &quot;F_LENR_sgra
1036              &quot;M_ext_sgra_b&quot;, &quot;r_ext_sgra_b&quot;,
1037              // [2] Pillars
1038              &quot;M_pil_d&quot;, &quot;r_pil_d&quot;, &quot;DPM_res_pil&quot;, &quot;F_LENR_pil&quo
1039              &quot;M_ext_pil_d&quot;, &quot;r_ext_pil_d&quot;,
1040              // [3] 26D
1041              &quot;M_res26&quot;, &quot;r_res26&quot;, &quot;N_layers_res26&quot;, &quot;T_sf_res26&
1042              &quot;M_ext_res26&quot;, &quot;r_ext_res26&quot;,
1043              // [4] UTe2
1044              &quot;M_ute2&quot;, &quot;r_ute2&quot;, &quot;B_ute2&quot;, &quot;f_topo_ute2&quot;,
1045              &quot;M_ext_ute2&quot;, &quot;r_ext_ute2&quot;,
1046              // [5] Anyons
1047              &quot;M_any&quot;, &quot;r_any&quot;, &quot;E_anyons_loc&quot;, &quot;delta_c_loc&quot
1048              &quot;M_ext_any&quot;, &quot;r_ext_any&quot;,
1049              // [6] Fe-56
1050              &quot;M_fe56&quot;, &quot;r_fe56&quot;, &quot;Z_fe56&quot;, &quot;A_fe56&quot;, &quot;
1051              &quot;M_ext_fe56&quot;, &quot;r_ext_fe56&quot;,
1052              // [7] Vac
1053              &quot;M_vac&quot;, &quot;r_vac&quot;, &quot;N_harmonics_vac&quot;, &quot;gamma_vac_loc&
1054              &quot;M_ext_vac&quot;, &quot;r_ext_vac&quot;,
1055              // metrics
1056              &quot;diversity_score&quot;, &quot;dynamic_score&quot;, &quot;scalability_score&quot;,
1057          };
1058          for (const char* n : names)
1059              ofs &lt;&lt; n &lt;&lt; &quot; = &quot; &lt;&lt; getVariable(n) &lt;&lt; &quot;\n&quot
1060          for (const auto&amp; kv : dynamic_params)
1061              ofs &lt;&lt; &quot;dynamic.&quot; &lt;&lt; kv.first &lt;&lt; &quot; = &quot; &lt;&lt; 
1062          if (logging_enabled)
1063              std::cout &lt;&lt; &quot;[LOG] State exported to: &quot; &lt;&lt; filename &lt;&lt; &q
1064      }
1065
1066      /// Cross-validate g_base[sys_idx] against an external UQFF module.
1067      /// Store the other module&#39;s g beforehand: setDynamicParameter(&quot;cross_g&quot;, val).
1068      template&lt;typename OtherModuleT&gt;
1069      double cross_validate(const OtherModuleT&amp; /*other*/, int sys_idx,
```

```
                           double /*t_seconds*/ = 0.0) const {
        double g_here  = compute_g_base(sys_idx);
        double g_other = 0.0;
        auto it = dynamic_params.find(&amp;quot;cross_g&amp;quot;);
        if (it != dynamic_params.end()) g_other = it-&amp;gt;second;
        double frac = (g_here != 0.0)
                    ? std::abs(g_other - g_here) / std::abs(g_here)
                    : std::numeric_limits&amp;lt;double&amp;gt;::quiet_NaN();
        if (logging_enabled)
            std::cout &amp;lt;&amp;lt; &amp;quot;[XVAL002] sys[&amp;quot; &amp;lt;&amp;lt; sys_idx &amp;lt;&amp;lt; &amp;quot;] g_here=
                      &amp;lt;&amp;lt; &amp;quot;  g_other=&amp;quot; &amp;lt;&amp;lt; g_other
                      &amp;lt;&amp;lt; &amp;quot;  frac_diff=&amp;quot; &amp;lt;&amp;lt; frac &amp;lt;&amp;lt; &amp;quot;\n&amp;quot;;
        return frac;
    }
};

#endif // UQFF_LEARNING_ASSESSMENT_002_H
```

— End of UQFFLearningAssessment_002.cpp —